

Lab-2: CS-2203 Artificial Intelligence Lab

Assignment No: 02

Date: Jan 15, 2026

Submission Deadline: [Jan 15, 2026 - 17:55 Hrs. IST]

Instructions for Submission

General Instructions

Some assignments may need to be submitted via Google Form. Please carefully follow the instructions below to ensure proper submission:

File Preparation

Submission Instructions:

1. Store all assignment-related files in a single folder and compress it into a `.rar` or `.zip` file. The compressed file must be named in the following format:

`roll-number_assign1.rar` or `roll-number_assign1.zip`

2. For example, if your roll number is 2201AI01, the compressed file should be named as:

`2201AI01_assign1.rar` or `2201AI01_assign1.zip`

3. Save each program using the filename format specified alongside each question.
4. If all assignments are completed within a single Jupyter Notebook, name the notebook file using the following format:

`CS2203_Lab2.ipynb`

5. When using a single notebook for multiple problems, clearly **segregate the code for each problem**. Each problem must be introduced using a **text cell** containing the problem statement before the corresponding code segment.
6. After completing the notebook, place it inside the assignment folder, compress the folder, and upload it by following the above naming instructions.

7. Additionally, submit a **PDF containing both the code and its output**. To generate the PDF:

- Open the Jupyter Notebook file.
- Navigate to **File** → **Print**.
- Save the file as a PDF using the same name as the notebook (e.g., `CS2203_Lab1.pdf`).

Submission Link

Upload your compressed file using the following Google Form link:

<https://forms.gle/8MwGDLGYSD8BH9Nj9>

Problem Statement : 1 (Save as 1.ipynb)

Write a **Python program from scratch** to implement the **complete forward propagation** of a deep, fully connected neural network with fixed architecture, fixed inputs, and fixed parameters. The program must compute and display all intermediate values clearly.

Network Architecture

The neural network has the following structure:

- Input Layer: 2 neurons
- Hidden Layer: 2 neurons with **Sigmoid activation**
- Output Layer: 1 neuron with **Sigmoid activation**
- Loss Function: Mean Squared Error (MSE)

Fixed Input and Hyperparameters

The input and learning parameters are fixed as:

```
x1 = 1
x2 = 1
Target output (y) = 0
```

Fixed Weights and Biases

Hidden Layer Parameters:

```
w11 = 0.15    w12 = 0.25
w21 = 0.20    w22 = 0.30
b1  = 0.35    b2  = 0.35
```

Output Layer Parameters:

```
v1 = 0.40
v2 = 0.45
bo = 0.60
```

Tasks (Strictly Coding Only)

Your Python program must perform the following tasks in the given order:

1. Compute the net input to each hidden neuron.
2. Apply the sigmoid activation function to obtain hidden layer outputs.
3. Compute the net input to the output neuron.
4. Apply the sigmoid activation function to obtain the final network output.
5. Compute the loss using the Mean Squared Error formula.
6. Print all intermediate values clearly, including:
 - Net input of each hidden neuron
 - Activation output of each hidden neuron
 - Net input of the output neuron
 - Final predicted output ($\hat{y}$)
 - Loss value

Mathematical Definitions

Use the following formulas:

- Sigmoid activation:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Mean Squared Error:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Expected Output (Reference Values)

The program output should approximately match the following values:

Hidden neuron 1 net input	0.70
Hidden neuron 2 net input	0.90
Hidden neuron 1 output	0.6682
Hidden neuron 2 output	0.7109
Output neuron net input	1.1060
Final output ($\hat{y}$)	0.7514
Loss	0.2823

Important Instructions

- Weights and biases must NOT be updated.
- Do NOT hard-code the final output values.
- Minor floating-point differences up to 10^{-3} are acceptable.

Problem Statement : 2 (Save as 2.ipynb)

Loss Function:

$$L(w) = (w - 2)^4$$

Initial weight:

$$w_0 = -1$$

You are required to check the behavior of **gradient descent** on the above loss function for different learning rates using python code.

Learning Rates

Perform exactly **5 iterations** of gradient descent for each of the following learning rates:

- (a) $\eta = 0.01$
- (b) $\eta = 0.1$
- (c) $\eta = 0.5$

Tasks

1. Compute the gradient $\frac{dL}{dw}$ analytically.
2. Starting from $w_0 = -1$, compute the updated value of w after each iteration for all three learning rates.
3. Tabulate the weight values obtained at each iteration.
4. For each learning rate, observe and describe:
 - Speed of convergence
 - Presence or absence of oscillations
 - Whether the updates overshoot the minimum
5. Identify which learning rate provides the best trade-off between stability and speed for this loss function.

Important Instructions

- Use the standard gradient descent update rule:

$$w_{k+1} = w_k - \eta \frac{dL}{dw}$$

- Numerical values should be rounded to four decimal places.